

Build a Subagentic RAG Assistant

Project assignment | Kestrel Systems Employee Help Desk | Follows the session on subagents, routers, parallel agents, handoffs and skills

Item	Details
What you build	A chat assistant that answers employee questions from three company policy PDFs (HR, IT, Finance). A supervisor agent sends each question to one or more specialist subagents, each searching only its own document, then combines their findings into one cited answer.
Level	Beginner to intermediate. You should know basic Python and have built one simple RAG app.
Time	About 12 to 14 hours, spread over 10 phases.
Stack	Python 3.11+, LangGraph, LangChain, ChromaDB, OpenAI GPT-4o and GPT-4o-mini, OpenAI text-embedding-3-small, Streamlit. LangSmith tracing is optional.
Data (provided)	Kestrel_HR_Policy_Handbook_v4.1.pdf, Kestrel_IT_Services_and_Security_Policy_v2.3.pdf, Kestrel_Travel_Expense_and_Relocation_Policy_v3.0.pdf
You submit	GitHub repository, README, 3 to 5 minute demo video, evaluation sheet, one-page comparison report.
Out of scope	Deployment, login, and access control. Run everything on your own machine.

1. The scenario

Kestrel Systems Pvt. Ltd. is a technology services company with offices in Pune (headquarters), Mumbai and Bengaluru. Employees ask the internal help desk hundreds of policy questions every month: leave balances, VPN problems, expense limits, relocation, resignation. The answers sit in three policy documents, each owned by a different department:

Document	Owner	Version and date	What it covers
HR Policy Handbook	HR Policy Team	v4.1, effective 1 April 2026	Working hours, hybrid and remote work, leave, transfers and relocation, onboarding, exit, grievances
IT Services & Information Security Policy	IT Services & Security	v2.3, effective 1 January 2026	Laptops, accounts, VPN, service desk and priorities, equipment during transfers, lost devices, exit, data and AI tool rules
Travel, Expense & Relocation Policy	Finance Controller	v3.0, effective 1 July 2026	Expense claims, travel class, daily limits by grade, approvals, relocation money, advances, final settlement

The help desk team wants an assistant that answers these questions correctly, shows exactly where each rule comes from, and admits when a question is not covered.

2. The problem the team faced

The team's first attempt was a standard RAG app: all three PDFs split into chunks, stored in one vector collection, and searched together for every question. In a two-week pilot it failed in four repeatable ways:

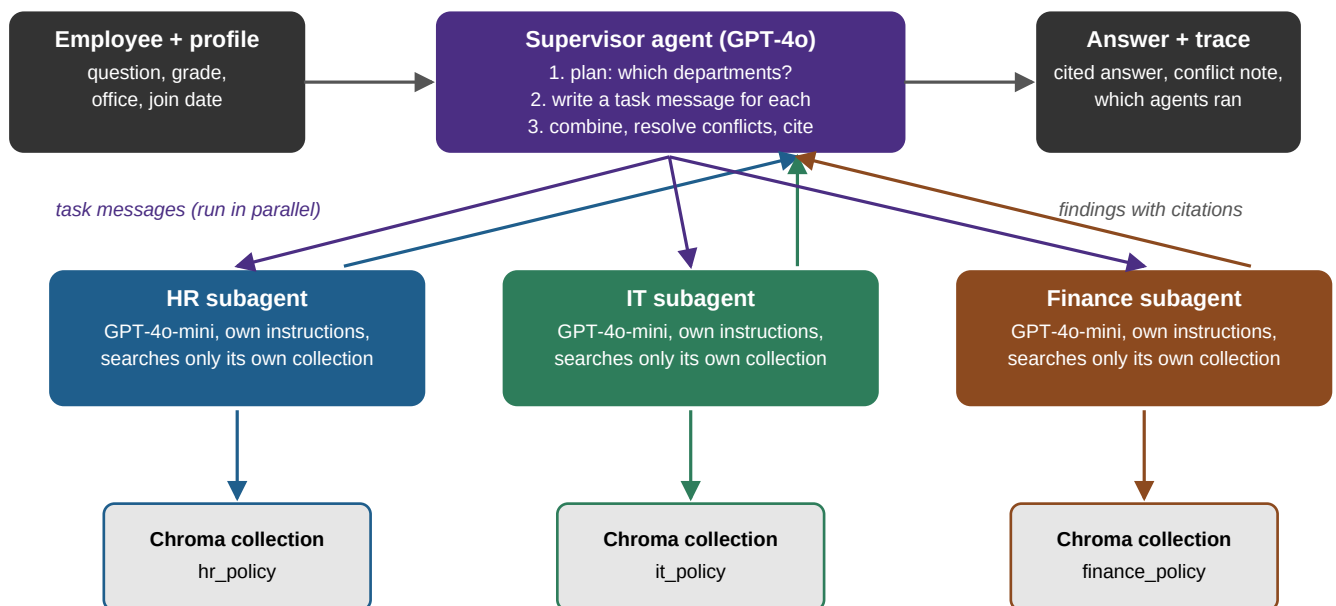
Failure	What happened in the pilot	Why it happened
Mixed-up rules	Asked about hotel stays during relocation, the assistant quoted business-travel hotel rules and relocation rules together as if they were one rule.	One search over all documents returned similar-looking chunks from different sections.
Missing parts	Asked "I'm relocating, what do I need to do?", it answered the Finance part and missed the IT desk setup deadline completely.	The top search results all came from one document; the others never reached the model.
Outdated rule	It quoted a relocation limit from the April HR handbook that the July Finance policy had replaced.	Nothing in the design compared documents or checked which rule is current.

Failure	What happened in the pilot	Why it happened
Invented answer	Asked about sabbaticals, which no policy covers, it described a sabbatical policy.	No rule forced it to say "not covered" when nothing relevant was found.

3. The plan: a subagentic RAG design

The team decided to split the work the same way the company splits ownership: one specialist per department, coordinated by one supervisor. This is the subagents pattern from the session. The flow for every question is:

1. The employee asks a question. Their profile (grade, office, joining date) is set in the app sidebar.
2. The **supervisor agent** decides which departments the question needs: none, one, two or all three.
3. For each department needed, the supervisor writes a complete **task message**: the exact question for that department plus any employee details that matter.
4. The chosen **subagents run at the same time**. Each one searches only its own Chroma collection and returns short findings with document, section and page for every rule.
5. The supervisor **combines the findings**, checks whether two documents disagree, applies the conflict rules, and writes one answer where every rule carries a citation.
6. If no department fits, or no subagent finds anything, the assistant says the question is not covered and suggests who to contact.



Why subagents, and not the other patterns?

Pattern	Fit for this project?	Reason
Single agent, one collection	No	This is the pilot that failed: mixed rules, missing parts.
Router	Partly	Works for one-department questions, but relocation and exit questions need two or three departments and one combined answer, with a conflict check across them.
Handoffs	No	Handoffs pass the chat to one agent at a time, so a three-department question would run in sequence and no single agent would see all findings to spot conflicts.
Skills	Not alone	Skills give one agent procedures, but all policy text would pile up in one workspace. Skills appear later as a stretch goal for answer formatting.
Subagents	Yes	Each department's text stays in its own workspace, departments run in parallel, and the supervisor sees every finding in one place before answering.

4. What you will learn

- Split PDFs by clause and keep useful metadata (document, version, effective date, section, page).
- Keep separate vector collections per data owner and search each one on its own.
- Build a subagent with narrow instructions, one search tool and a fixed output format.
- Build a supervisor that plans, writes complete task messages and runs subagents in parallel with LangGraph.
- Detect and resolve conflicts between documents, and cite every rule.
- Measure routing accuracy, answer accuracy, time and token cost, and compare against a single-agent baseline.

5. Get to know the data before you build

Open all three PDFs and read them once, fully. They are short on purpose. While reading, look for these four things, because your system must handle each one:

What to look for	Example you will find	What your system must do
Cross-references between documents	HR Handbook 4.3 says relocation money is defined in the Finance policy, Section 6.	Send relocation questions to both HR and Finance, not only HR.
The same topic in two documents	Temporary accommodation on relocation appears in the HR Handbook and in the Finance policy.	Notice when two findings cover the same topic with different values.
Rules that depend on the employee	Hotel and meal limits depend on grade; notice period depends on grade; some limits depend on metro or non-metro city.	Pass grade and office from the profile into task messages; show calculations.
Repeated header and footer lines	Every page carries the company name, document title, document ID and page number.	Remove these lines before splitting, so they do not pollute chunks.

6. Tools and setup

Tool	Used for	Notes
Python 3.11 or later	Everything	Create a virtual environment for the project.
LangChain	PDF loading, text splitting, OpenAI and Chroma connectors	Use the pypdf-based PDF loader.
LangGraph	Supervisor and subagent workflow, parallel runs	You will use state, nodes, edges and the Send feature for parallel fan-out.
ChromaDB	Three persistent vector collections	Store on disk so ingestion runs once.
OpenAI GPT-4o	Supervisor: planning and final answer	Needs careful reasoning across findings.
OpenAI GPT-4o-mini	The three subagents	Cheaper and faster; their job is narrow.
text-embedding-3-small	Embeddings for all chunks and queries	Use the same model for all three collections.
Streamlit	Chat UI with sidebar profile and trace panel	Gradio is an acceptable alternative.
LangSmith (optional)	Viewing traces of every run	Very useful in Phases 6 to 10.

Keep your API key safe

Store the OpenAI key in a .env file, load it at startup, and add .env to .gitignore. Repositories containing a key receive zero marks for code quality.

Suggested project layout

Folder or file	Contents
data/	The three policy PDFs

Folder or file	Contents
ingest	Reading, cleaning, splitting and loading into Chroma (Phases 1 and 2)
subagents	Shared subagent builder plus HR, IT and Finance instructions (Phases 3 and 4)
supervisor	Planning, synthesis and conflict rules (Phases 5 and 7)
graph	The LangGraph workflow that connects everything (Phase 6)
app	Streamlit UI (Phase 8)
eval	Test questions, evaluation runner, results sheet, baseline (Phases 9 and 10)
README.md	Setup steps, design decisions, results summary

7. Build it, phase by phase

Work through the phases in order. Each phase ends with a checkpoint. Do not start the next phase until the checkpoint passes; later phases depend on earlier ones working correctly.

Phase 0 · Set up the project (about 30 minutes)

Goal: a clean project that can call OpenAI and create a Chroma collection.

1. Create the project folder, virtual environment and the layout above.
2. Install the libraries from Section 6 and record them in a requirements file.
3. Put the three PDFs in data/ and your key in .env.
4. Make one test call to GPT-4o-mini and one embedding call to confirm the key works.

Checkpoint: you are done with this phase when...

- A one-line test prompt returns a reply, and an embedding call returns a list of numbers.

Phase 1 · Read, clean and split the PDFs (about 1.5 hours)

Goal: one chunk per policy clause or table, each with metadata that tells you exactly where it came from.

1. Extract the text page by page, keeping the page number.
2. Remove the repeated header and footer lines on every page (company name, document title, document ID line, page number).
3. Split by clause number. Every clause starts with a number such as 4.4 or 6.2. One clause becomes one chunk. Keep a clause and the table directly under it in the same chunk.
4. Give every chunk this metadata: department (hr, it, finance), document ID, document title, version, effective date, section number, section heading, page number.
5. If a clause is very long, split it further with a character splitter, keeping the same metadata.

Checkpoint: you are done with this phase when...

- You can print the chunk count for each document (roughly 30 to 35 per document).
- The chunk for Finance section 6.4 contains the full clause text and shows version 3.0, effective date 1 July 2026, and its page number.
- No chunk contains the header or footer text.

Common mistake

Splitting by a fixed number of characters only. This cuts clauses in half, loses section numbers, and makes correct citations impossible.

Phase 2 · Create three vector collections (about 1 hour)

Goal: three separate Chroma collections, one per department, that return the right clause for simple searches.

1. Create collections named hr_policy, it_policy and finance_policy.

2. Embed each document's chunks into its own collection, using text-embedding-3-small for all three.
3. Give every chunk a stable ID built from document ID and section number, so re-running ingestion replaces chunks instead of duplicating them.
4. Save the collections to disk. Ingestion should run once, not every time the app starts.

Test each collection with direct searches before building any agent:

Collection	Test search	Expected section in the top 3
hr_policy	how many earned leave days per year	3.1
hr_policy	notice period for a manager grade	6.1
it_policy	VPN keeps disconnecting	5.3
it_policy	lost laptop	7.1
finance_policy	meal allowance per day	4 (daily limits table)
finance_policy	repay relocation allowance after resigning	6.6

Checkpoint: you are done with this phase when...

- All six test searches return the expected section in the top 3 results.
- Running ingestion twice leaves the chunk counts unchanged.

Phase 3 · Build your first subagent: HR (about 1.5 hours)

Goal: an HR specialist that answers only from the HR collection, cites everything, and says when something is not covered.

A subagent has three parts. Design each one deliberately:

Part	What to build for the HR subagent
Instructions	Role: HR policy specialist. Use only the retrieved HR clauses. Quote the section number and page for every rule. State the document version and effective date. If a rule depends on grade or probation, say so. If nothing relevant is found, say "not covered in the HR Handbook". Never guess.
One tool	A search tool over hr_policy only, returning the top 4 chunks with their metadata. The subagent may call it more than once with different wording.
Fixed output format	Use structured output with these fields: department; covered (yes or no); findings, where each finding has the rule in plain words, section, page, document ID, version and effective date; and gaps, for parts of the task the HR documents do not answer.

Remember what a subagent can see: **only its own instructions and the task message**. It never sees the chat history or the other subagents. Test it directly with task messages like these:

Task message sent to the HR subagent	A correct result contains
Employee grade G4, confirmed. What is the notice period, and can earned leave be taken during notice?	60 days (6.1); earned leave during notice only with HR Head approval (3.3)
Employee in probation, joined 2 months ago. Can they take earned leave next week?	No; earned leave accrues but can be taken only after confirmation (3.2)
What is the company policy on sabbatical leave?	covered: no, with a gap stating sabbatical is not in the HR Handbook

Checkpoint: you are done with this phase when...

- All three test task messages return the expected result in the fixed format.
- Every finding has a section number and page number.

Phase 4 · Build the IT and Finance subagents (about 1 hour)

Goal: two more subagents built the same way, each with instructions suited to its department.

1. Reuse the same subagent design; change only the instructions and the collection.
2. IT subagent: list troubleshooting steps in the exact order the policy gives; always include the ticket priority and response time when a ticket is involved.
3. Finance subagent: when an amount depends on grade or city, show the calculation in steps (for example: non-metro limit = 75% of INR 4,000 = INR 3,000). Always state whether the city is metro.
4. Test each with two task messages of your own and one question from the wrong department. The wrong-department question should come back as not covered.

Checkpoint: you are done with this phase when...

- The IT subagent returns the three VPN steps in order and the P3 response time.
- The Finance subagent calculates a G2 non-metro hotel limit correctly, with the steps shown.
- An HR question sent to the Finance subagent returns covered: no.

Phase 5 · Build the supervisor's planning step (about 1.5 hours)

Goal: a supervisor that decides which departments a question needs and writes a complete task message for each.

The supervisor reads the question, the last few chat messages and the employee profile. It returns a plan with: the departments needed (zero to three), one task message per department, and a one-line reason. Use structured output so your code can read the plan reliably.

Weak task message	Strong task message
"Check relocation stuff for this user."	"Employee: grade G5, Pune office, joined March 2024. They are transferring to the Mumbai office next month. List every financial relocation benefit they get: allowance amount, travel, temporary accommodation limit (nights and nightly amount) and household goods. Include section numbers."

1. Write the supervisor instructions: its departments and what each covers (use Section 1 of this document), the rule that a question can need more than one department, and the rule that every task message must stand on its own because subagents see nothing else.
2. Always copy the profile details that matter (grade, office, joining date) into the task message.
3. If the question mentions a city, say whether it is a metro city in the Finance task message.
4. If no department fits, return an empty plan. Do not force a question into a department.
5. Test planning on its own, before connecting subagents, with the 16 test questions in Section 8. Compare the departments chosen with the expected column.

Checkpoint: you are done with this phase when...

- At least 14 of the 16 test questions get the expected set of departments.
- The share-price question returns an empty plan.

Phase 6 · Connect everything in LangGraph and run subagents in parallel

(about 1 hour)

Goal: one workflow where all chosen subagents run at the same time and their findings are collected in one place.

1. Define the workflow state: question, profile, chat history, plan, a list of subagent findings, final answer, and timing and token figures.
2. Nodes: planning, one subagent node, and synthesis. Use the Send feature to start one subagent run per department in the plan, all in parallel. Give the findings list a reducer that adds items, so parallel results are merged instead of overwriting each other.
3. Empty plan: skip the subagents and go straight to the out-of-scope reply.
4. Failure handling: if a subagent fails or takes longer than 30 seconds, record a gap saying that department could not be checked, and continue with the rest.
5. Record the start and end time and the tokens used for each subagent.

Checkpoint: you are done with this phase when...

- The relocation question runs three subagents, and the total time is close to the slowest single subagent, not the sum of all three.
- Switching off the IT subagent on purpose produces an answer that clearly states the IT part could not be checked.

Phase 7 · Combine findings, cite sources and handle conflicts (about 1.5 hours)

Goal: one clear answer where every rule is cited and disagreements between documents are resolved and shown.

The synthesis step receives all findings and writes the final answer. Build these rules into its instructions:

Rule	What the supervisor must do
Cite every rule	Each rule in the answer ends with a citation such as [KSPL-FIN-POL-003, section 6.2].
Use only findings	No number, name or deadline may appear unless a subagent reported it.
Show calculations	Carry through any calculation a subagent showed; do not do new arithmetic silently.
Detect conflicts	If two findings cover the same topic with different values, treat it as a conflict.
Resolve in this order	1) A clause that explicitly replaces earlier limits wins. 2) The Finance policy prevails on financial benefits (Finance 1.3). 3) Otherwise the newer effective date wins. 4) If still unclear, show both values and tell the employee to confirm with HR.
Show the conflict	Add a short note: which documents disagree, the two values, which one applies, and why.
Report gaps	List anything no subagent could answer, with the contact from HR Handbook 7.4 where it helps.

Checkpoint: you are done with this phase when...

- The relocation and temporary accommodation questions give the current limit and a conflict note that names both documents.
- The sabbatical question says it is not covered and suggests the HR Business Partner, with no invented details.
- Every rule in every answer has a citation.

Phase 8 · Build the user interface (about 1.5 hours)

Goal: a chat app that shows both the answer and how the system reached it.

1. Chat area: the employee types questions and sees answers with citations. Keep chat history for follow-up questions.
2. Sidebar profile: name, grade (G1 to G7), office (Pune, Mumbai, Bengaluru) and joining date.
3. "Behind this answer" panel, collapsed by default, showing: the supervisor's plan and reason; each task message; each subagent's findings; time and tokens per subagent; total time and an estimated cost.
4. Show a clear badge when a conflict was found.

Checkpoint: you are done with this phase when...

- A reviewer can see, for any answer, which subagents ran and what each one returned.
- Changing grade in the sidebar from G2 to G6 changes the hotel limit in the answer.

Phase 9 · Evaluate against the test set (about 1.5 hours)

Goal: numbers that show how well your assistant works, not just a demo that looks good.

1. Run all 16 questions in Section 8 with the profiles given.
2. For each question, record in your evaluation sheet: departments expected vs chosen; key points expected vs present; citations correct (yes or no); conflict flagged where expected; time; total tokens.
3. Calculate: routing accuracy, key-point coverage, citation accuracy, out-of-scope handling, average time and average tokens per question.

- Look at every failure. Decide whether it came from splitting, search, the subagent, planning or synthesis, and fix the right part.

Targets: routing accuracy of at least 90%; key-point coverage of at least 85%; 100% of rules cited; both out-of-scope questions handled correctly; the accommodation conflict flagged in both questions where it appears.

Phase 10 · Compare with a single-agent baseline (about 1 hour)

Goal: evidence of what the subagent design gained, and what it cost.

- Build the pilot design from Section 2: one collection with all chunks and one GPT-4o agent with one search tool. Keep the same splitting and embeddings.
- Run the same 16 questions and fill the same evaluation sheet.
- Write a one-page comparison: accuracy, citations, conflicts caught, time and tokens for both designs, with two example questions where the results differ.

Link back to the session

The session showed that subagents cost an extra model call and more tokens for simple requests, but win on multi-domain questions. Your comparison should show whether that held for Kestrel's questions.

8. Acceptance test set

Set the profile as shown before each question. "Any" means any profile. The last column lists the points a correct answer must contain; wording can differ.

#	Question	Profile	Departments	A correct answer must contain
1	How many earned leaves do I get a year, and how many can I carry forward?	Any	HR	18 days, 1.5 per month; carry forward up to a 30-day balance
2	My VPN keeps disconnecting. What should I do?	Any	IT	Update client, switch network, reset profile, in that order; then P3 ticket with logs; 1 business day
3	What is my daily meal limit on a business trip?	G4	Finance	INR 1,500 per day, all cities
4	How long do I have to submit an expense claim?	Any	Finance	Within 30 days; rejected after 60 days
5	What is my hotel limit per night in Nashik?	G2	Finance	Nashik is non-metro; 75% of INR 4,000 = INR 3,000
6	I lost my laptop at the airport. What now?	Any	IT	P1 ticket within 24 hours; police complaint copy; remote wipe; 25% of book value if negligent
7	Can I paste customer data into ChatGPT to summarise it?	Any	IT	Customer data is Confidential; never in public or free tools; only Kestrel Copilot or the ChatGPT Enterprise workspace
8	I join next Monday. What documents do I bring, and when do I get my laptop?	New joiner	HR, IT	Document list; laptop on day 1 and accounts before joining if HR requested 5 working days ahead, otherwise within 2 working days
9	Can I work from Goa for 3 weeks?	G3, Pune	HR, IT	Up to 20 working days a year with manager approval, 5 working days ahead; VPN required; public Wi-Fi only with VPN
10	I want to work from Dubai for 2 weeks. Is that allowed?	G4	HR, IT	HR Business Partner and IT Security approval; maximum 15 working days a year; Overseas Access ticket 10 working days before; sanctioned-country check
11	I'm resigning. What happens with notice, my laptop and final settlement?	G4	HR, IT, Finance	60 days notice; EL in notice needs HR Head approval; up to 30 days EL encashed; return assets on last day; accounts off at 18:00; IT clearance to Finance in 2 working days; settlement within 45 days

#	Question	Profile	Departments	A correct answer must contain
12	I'm moving from Pune to the Mumbai office next month. What do I get and what must I do?	G5, 2 years' service	HR, IT, Finance	Form HR-12 30 days ahead; 3 days relocation leave; INR 75,000; travel for dependents; current accommodation limit with conflict note; goods up to INR 60,000, lowest of 3 quotes; laptop in cabin bag; Desk Setup ticket 10 working days ahead; new access card
13	I relocated 8 months ago and now I'm resigning. Do I repay anything?	G5	Finance (HR optional)	4 unserved months; 4/12 of INR 75,000 = INR 25,000; recovered in final settlement
14	How many nights of temporary accommodation do I get when relocating?	Any	HR, Finance	The current limit with the clause that sets it; mentions the other document's value and why it no longer applies
15	What is the policy on sabbatical leave?	Any	HR	Not covered; suggest the HR Business Partner; no invented details
16	What is Kestrel's share price today?	Any	None	Politely out of scope; says what the assistant can help with; no subagent called

9. What to submit

Deliverable	Requirements
GitHub repository	Code in the layout from Section 6; requirements file; no API keys.
README.md	Setup and run steps; one diagram of your design; design decisions (chunking, top-k, models, conflict rules); results summary.
Demo video, 3 to 5 minutes	Show questions 2, 9, 12 and 16, opening the "Behind this answer" panel each time.
Evaluation sheet	All 16 questions for both designs, with the metrics from Phase 9.
Comparison report, one page	Subagent design vs single-agent baseline, with your conclusion.

10. How you will be graded

Area	Marks	Full marks means
Ingestion and collections	10	Clause-level chunks, full metadata, headers removed, three collections
Subagents	20	Narrow instructions, own collection only, fixed output, honest "not covered"
Supervisor planning	15	Correct departments; complete, standalone task messages
Parallel workflow	10	Subagents run together; failures handled without crashing
Synthesis and conflicts	15	Every rule cited; conflicts found, resolved and explained
User interface	10	Profile sidebar and a clear "Behind this answer" panel
Evaluation and comparison	15	Complete sheet for both designs; honest analysis of failures
Code quality and README	5	Clean structure, clear setup, no secrets
Total	100	

11. Stretch goals (optional, up to 10 bonus marks)

Stretch goal	What to build	Pattern practised
Answer formatting skill	Write a SKILL.md called relocation-checklist that tells the supervisor how to format relocation answers as a dated checklist. Load it only for relocation questions.	Skills
Human review for conflicts	Pause the workflow when a conflict cannot be resolved by the rules, show it to a reviewer in the UI, and continue after their choice (LangGraph interrupt).	Human-in-the-loop

Stretch goal	What to build	Pattern practised
Router comparison	Replace the planning step with a simple router that picks exactly one department, run the test set, and compare with your subagent results.	Router
Voting check on money answers	For answers that contain amounts, run two independent checker calls against the findings and flag the answer if they disagree.	Parallel voting
Handoff to a human	Add a "raise a ticket" path that passes the question, profile and findings to a human queue when the employee is not satisfied.	Handoffs

12. When things go wrong

What you see	Likely cause	What to try
Right department, wrong clause	Chunks too large or header text inside chunks	Check Phase 1 splitting; re-run the Phase 2 test searches
Subagent answers from general knowledge	Instructions too loose	Require a section number for every finding; reject findings without one
Relocation answer misses IT	Supervisor instructions do not describe what IT covers	Add IT's relocation coverage (equipment, desk setup, access card) to the supervisor's department list
Parallel results overwrite each other	Findings list has no reducer	Add a reducer that appends
Conflict never flagged	Findings lack effective date or version	Include version and date in every finding; restate the conflict rules in the synthesis instructions
Answers change between runs	High temperature	Set temperature to 0 for planning and subagents

13. Reflection questions (answer in your README)

1. Which question showed the biggest difference between the subagent design and the baseline, and why?
2. What did the subagent design cost you in time and tokens on single-department questions?
3. Where did your supervisor choose the wrong departments, and what did you change?
4. Would a router have been enough for Kestrel? Use your evaluation numbers to answer.
5. If a fourth department (Legal) added its own policy, which parts of your system would change?